

LinearRegression#

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

Description:

Ordinary least squares Linear Regression. LinearRegression fits a linear model with coefficients $w = (w_1, \dots, w_p)$ to minimize the residual sum of squares between the observed targets in the dataset, and the targets predicted by the linear approximation. Whether to calculate the intercept for this model. If set to False, no intercept will be used in calculations (i.e. data is expected to be centered). If True, X will be copied; else, it may be overwritten.

Function Signature

```
class sklearn.linear_model.LinearRegression(*,  
fit_intercept=True, copy_X=True, tol=1e-06, n_jobs=None,  
positive=False)[source]#
```

Parameters:

fit_interceptbool, default=True

copy_Xbool, default=True

tolfloat, default=1e-6

```
class sklearn.linear_model.LinearRegression(*,  
fit_intercept=True, copy_X=True, tol=1e-06, n_jobs=None,  
positive=False)[source]#
```

Parameters:

fit_interceptbool, default=True

copy_Xbool, default=True

tolfloat, default=1e-6

```
fit(X, y, sample_weight=None)[source]#
```

Parameters:

X{array-like, sparse matrix} of shape (n_samples,
n_features)

yarray-like of shape (n_samples,) or (n_samples, n_targets)

sample_weightarray-like of shape (n_samples,), default=None

Parameters

tolfloat, default=1e

The precision of the solution (coef_) is determined by tol which specifies a different convergence criterion for the lsqr solver. tol is set as atol and btol of `scipy.sparse.linalg.lsqr` when fitting on sparse training data. This parameter has no effect when fitting on dense data. Added in version 1.7.

coef_array of shape (n_features,) or (n_targets, n_features)

Estimated coefficients for the linear regression problem. If multiple targets are passed during the fit (y 2D), this is a 2D array of shape (n_targets, n_features), while if only one target is passed, this is a 1D array of length n_features.

intercept_float or array of shape (n_targets,)

Independent term in the linear model. Set to 0.0 if `fit_intercept = False`.

X{array

Training data.

yarray

Target values. Will be cast to X's dtype if necessary.

`sample_weightarray`

Individual weights for each sample. Added in version 0.17: parameter `sample_weight` support to `LinearRegression`.

`paramsdict`

Parameter names mapped to their values.

`Xarray`

Samples.

`Xarray`

Test samples. For some estimators this may be a precomputed kernel matrix or a list of generic objects instead with shape `(n_samples, n_samples_fitted)`, where `n_samples_fitted` is the number of samples used in the fitting for the estimator.

`yarray`

True values for X.

`sample_weightarray`

Sample weights.

`**paramsdict`

Estimator parameters.

Code Examples

```
>>> import numpy as np
>>> from sklearn.linear_model import LinearRegression
>>> X = np.array([[1, 1], [1, 2], [2, 2], [2, 3]])
>>> # y = 1 * x_0 + 2 * x_1 + 3
>>> y = np.dot(X, np.array([1, 2])) + 3
>>> reg = LinearRegression().fit(X, y)
>>> reg.score(X, y)
1.0
>>> reg.coef_
array([1., 2.])
>>> reg.intercept_
np.float64(3.0)
>>> reg.predict(np.array([[3, 5]]))
array([16.])
```

Notes & Warnings

NOTE

See also `Ridge` regression addresses some of the problems of Ordinary Least Squares by imposing a penalty on the size of the coefficients with L_2 regularization. `Lasso` The Lasso is a linear model that estimates sparse coefficients with L_1 regularization. `ElasticNet` Elastic-Net is a linear regression model trained with both L_1 and L_2 -norm regularization of the coefficients.

Detailed Documentation

Documentation

Ordinary least squares Linear Regression.

`LinearRegression` fits a linear model with coefficients $w = (w_1, \dots, w_p)$ to minimize the residual sum of squares between the observed targets in the dataset, and the targets predicted by the linear approximation.

Whether to calculate the intercept for this model. If set to `False`, no intercept will be used in calculations (i.e. data is expected to be centered).

If `True`, `X` will be copied; else, it may be overwritten.

The precision of the solution (`coef_`) is determined by `tol` which specifies a different convergence criterion for the `lsqr` solver. `tol` is set as `atol` and `btol` of `scipy.sparse.linalg.lsqr` when fitting on sparse training data. This parameter has no effect

when fitting on dense data.

Added in version 1.7.

The number of jobs to use for the computation. This will only provide speedup in case of sufficiently large problems, that is if firstly `n_targets > 1` and secondly `X` is sparse or if `positive` is set to `True`. `None` means `1` unless in a `joblib.parallel_backend` context. `-1` means using all processors. See Glossary for more details.

When set to `True`, forces the coefficients to be positive. This option is only supported for dense arrays.

For a comparison between a linear regression model with positive constraints on the regression coefficients and a linear regression without such constraints, see `Non-negative least squares`.

Added in version 0.24.

Estimated coefficients for the linear regression problem. If multiple targets are passed during the fit (`y` 2D), this is a 2D array of shape `(n_targets, n_features)`, while if only one target is passed, this is a 1D array of length `n_features`.

Rank of matrix `X`. Only available when `X` is dense.

Singular values of `X`. Only available when `X` is dense.

Independent term in the linear model. Set to `0.0` if `fit_intercept = False`.

Number of features seen during fit.

Added in version 0.24.

Names of features seen during fit. Defined only when `X` has feature names that are all

strings.

Added in version 1.0.

Ridge regression addresses some of the problems of Ordinary Least Squares by imposing a penalty on the size of the coefficients with L_2 regularization.

The Lasso is a linear model that estimates sparse coefficients with L_1 regularization.

Elastic-Net is a linear regression model trained with both L_1 and L_2 -norm regularization of the coefficients.

From the implementation point of view, this is just plain Ordinary Least Squares (`scipy.linalg.lstsq`) or Non Negative Least Squares (`scipy.optimize.nnls`) wrapped as a predictor object.

Target values. Will be cast to X's dtype if necessary.

Individual weights for each sample.

Added in version 0.17: parameter `sample_weight` support to `LinearRegression`.

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

A `MetadataRequest` encapsulating routing information.

Get parameters for this estimator.

If True, will return the parameters for this estimator and contained subobjects that are estimators.

Parameter names mapped to their values.

Predict using the linear model.

Returns predicted values.

Return coefficient of determination on test data.

The coefficient of determination, R^2 , is defined as $(1 - \frac{u}{v})$, where u is the residual sum of squares $((y_{\text{true}} - y_{\text{pred}})^2).sum()$ and v is the total sum of squares $((y_{\text{true}} - y_{\text{true.mean}}())^2).sum()$. The best possible score is 1.0 and it can be negative (because the model can be arbitrarily worse). A constant model that always predicts the expected value of y , disregarding the input features, would get a R^2 score of 0.0.

Test samples. For some estimators this may be a precomputed kernel matrix or a list of generic objects instead with shape $(n_{\text{samples}}, n_{\text{samples_fitted}})$, where $n_{\text{samples_fitted}}$ is the number of samples used in the fitting for the estimator.

R^2 of `self.predict(X)` w.r.t. y .

The R^2 score used when calling `score` on a regressor uses `multioutput='uniform_average'` from version 0.23 to keep consistent with default value of `r2_score`. This influences the `score` method of all the multioutput regressors (except for `MultiOutputRegressor`).

Configure whether metadata should be requested to be passed to the fit method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

True: metadata is requested, and passed to fit if provided. The request is ignored if metadata is not provided.

False: metadata is not requested and the meta-estimator will not pass it to fit.

None: metadata is not requested, and the meta-estimator will raise an error if the user provides it.

str: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Metadata routing for `sample_weight` parameter in fit.

Set the parameters of this estimator.

The method works on simple estimators as well as on nested objects (such as Pipeline). The latter have parameters of the form `<component>__<parameter>` so that it's possible to update each component of a nested object.

Estimator parameters.

Configure whether metadata should be requested to be passed to the score method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config`). Please check the User Guide

on how the routing mechanism works.

The options for each parameter are:

True: metadata is requested, and passed to score if provided. The request is ignored if metadata is not provided.

False: metadata is not requested and the meta-estimator will not pass it to score.

None: metadata is not requested, and the meta-estimator will raise an error if the user provides it.

str: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Metadata routing for `sample_weight` parameter in `score`.

Principal Component Regression vs Partial Least Squares Regression

Plot individual and voting regression predictions

Failure of Machine Learning to infer causal effects

Comparing Linear Bayesian Regressors

Non-negative least squares

Ordinary Least Squares and Ridge Regression

Robust linear model estimation using RANSAC

Robust linear estimator fitting

Face completion with a multi-output estimators

Plotting Cross-Validated Predictions

Underfitting vs. Overfitting

Using KBinsDiscretizer to discretize continuous features

© Copyright 2007 - 2025, scikit-learn developers (BSD License).